Lecture 9: Kernel methods and SGD

Machine Learning 2026

Federico Chiariotti (federico.chiariotti@unipd.it)



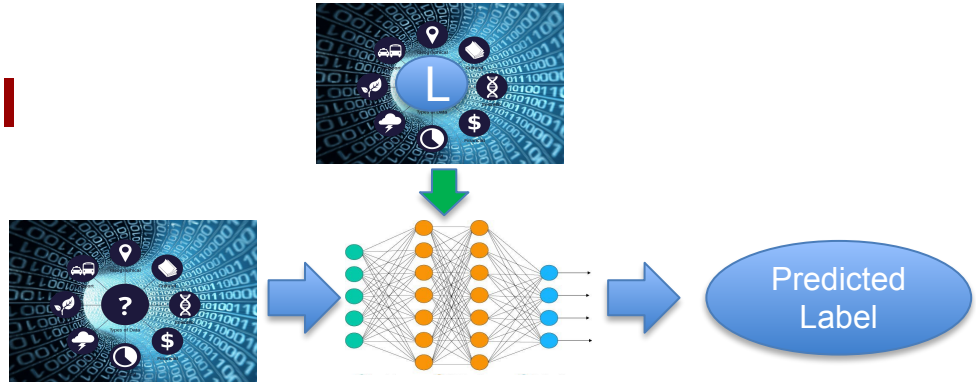
Università degli Studi di Padova

Recap



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Supervised learning model



- Prediction rule $h : X \rightarrow Y$
  - The learner's output (hypothesis)
  - $A(S)$: hypothesis produced by algorithm A when it is fed training set S
- Data generation model
  - D is a distribution over X **(unknown to the machine learning algorithm)**
  - Instances are labeled according to $f : X \rightarrow Y$ **(unknown to the ML algorithm)**
  - Training set: sampling according to $D, y_i = f(x_i)\ \forall x_i \in S$
- Success metric
  - Classifier error: probability of predicting the wrong label over D

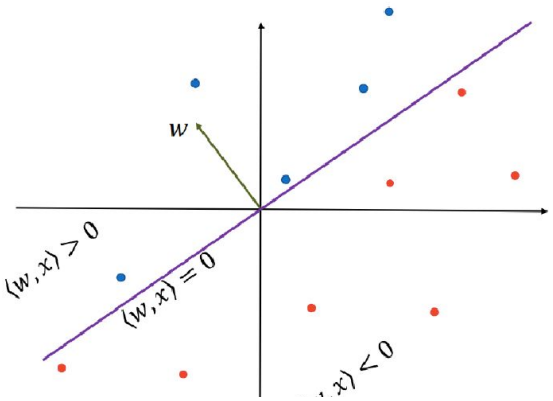
Classification: halfspaces

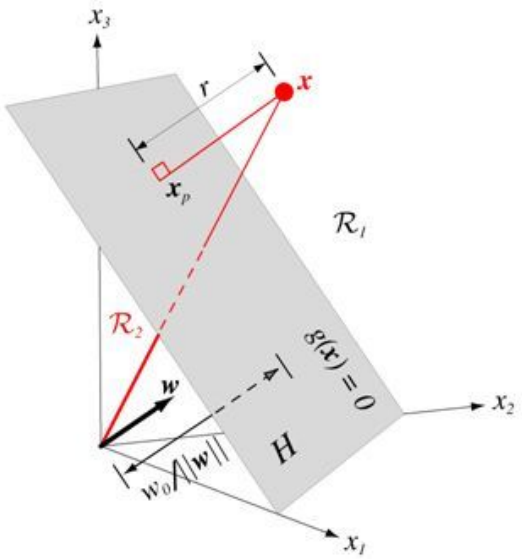
Domain: $X \subset \mathbb{R}^d$

Labels: $Y = \{-1, 1\}$

Loss: 0-1

Hypothesis class: $H = \text{sign} \circ L_d \qquad h_{\mathbf{w}'}(\mathbf{x}') = \text{sign}(\langle \mathbf{w}', \mathbf{x}' \rangle)$





2D: a line divides the two regions (d=2, d'=3)

3D: a plane divides the two regions (d=3, d'=4)

The perceptron algorithm

Start: $\mathbf{w}^{(0)} = (0, \ldots, 0)$

Repeat until convergence:

1. Find a sample with $\text{sign}(\langle \mathbf{w}^{(i)}, \mathbf{x}_m \rangle) = -y_m$
2. Compute the new $\mathbf{w}^{(i+1)} = \mathbf{w}^{(i)} + \alpha \mathbf{x}_m y_m$

Parameter α is the learning rate (if omitted, it is set to 1)

Termination depends on realizability: if the training set is not linearly separable, the perceptron will go on forever

Perceptron: implementation notes

- ➔ Normalizing the input can reduce convergence time (the perceptron may swing a lot if one dimension has a much wider dynamic range)
- ➔ Randomizing the selection of the **single** sample to use at each iteration can reduce the risk of unlucky orderings (i.e., a lot of misleading samples in the first positions)
- ➔ **Pocket algorithm**: compute the error for each iteration, keep the best one so that we have an approximate solution if the set is not linearly separable
- ➔ The perceptron will reach the ERM solution, but depending on the training, it may reach *any* ERM solution



Primal and dual problems

We can define a Lagrangian dual problem by turning a constrained problem into an unconstrained one.

Each constraint becomes a penalty in the objective function, with an associated Lagrange multiplier

The solution to the dual problem is *better* than the solution to the primal problem (but it might not be valid)



Example: primal and dual problems

Problem: minimize $f(x, y) = (x + y)^2$ such that $x^2 + y^2 \leq 1$

We move to the unconstrained problem

$$f'(x, y) = (x + y)^2 + \lambda(x^2 + y^2 - 1)$$

This is still a convex function! We can find the minimum by setting the gradient to 0. By gradually increasing the multiplier, we get closer and closer to the primal problem

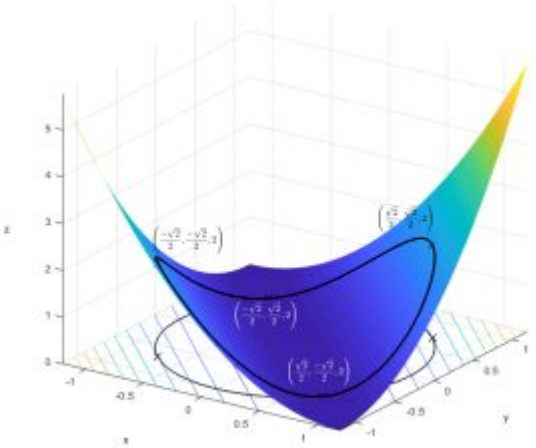


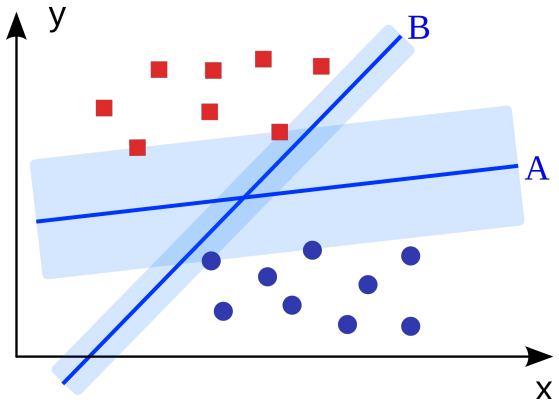
Image: Wikimedia Foundation

Margins

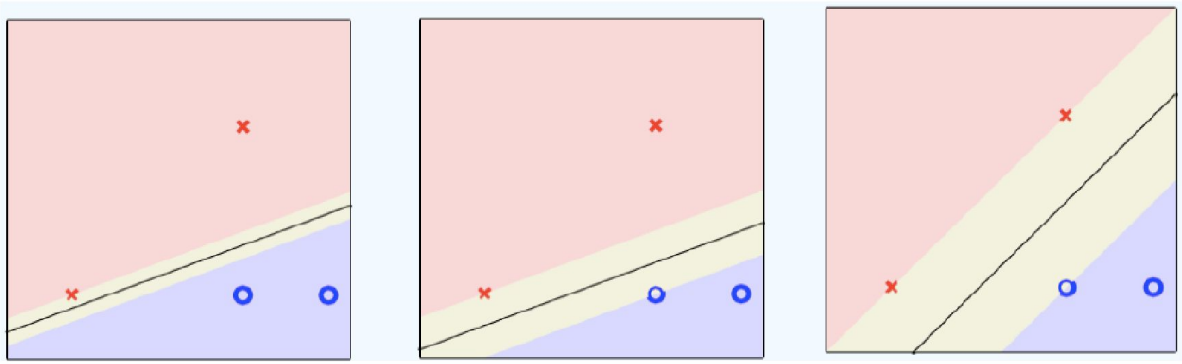
Let us assume realizability (i.e., the data is linearly separable).

We can define the **margin** as the minimum distance between points in the training set and the separating hyperplane

Imagine there is some “noise” on the inputs: a larger margin can tolerate points that are displaced farther



Defining margins



Given a hyperplane defined by $d(\mathbf{x}, L) = \min\{||\mathbf{x} - \mathbf{v}|| : \mathbf{v} \in L\}$ and a point $\mathbf{x}$, the distance between them is

$$L = \{\mathbf{v} : \langle \mathbf{v}, \mathbf{w} \rangle + b = 0\}$$

Hard SVMs

A hard SVM tries to maximize the margin (only on linearly separable data)

Optimization problem:

$$\arg\max_{(\mathbf{w},b):||\mathbf{w}||=1} \min_{\mathbf{x}_i \in S} |\langle \mathbf{w}, \mathbf{x}_i \rangle + b|$$

such that $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) > 0 \forall (x_i, y_i) \in S$

Equivalent formulation for separable data:

$$\arg\max_{(\mathbf{w},b):||\mathbf{w}||=1} \min_{(x_i,y_i) \in S} y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$$

An equivalent convex problem

We can find an equivalent formulation, which is a convex problem:

$$\arg\min_{\mathbf{w},b} ||\mathbf{w}||^2$$

$$\text{such that} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$$

This is a convex problem, which we can solve using standard tools (e.g., Lagrange multipliers or GD)

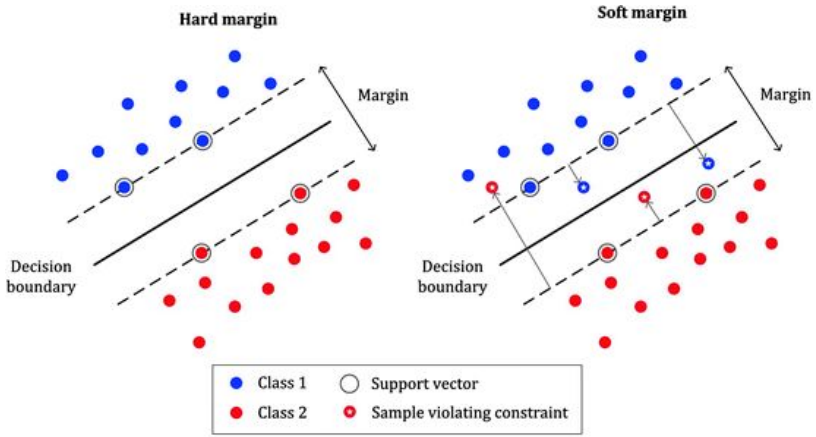
The soft SVM problem

$$\arg\min_{\mathbf{w},b,\xi} \left(\lambda ||\mathbf{w}||^2 + \frac{1}{m}\sum_{i=1}^{m} \xi_i\right)$$

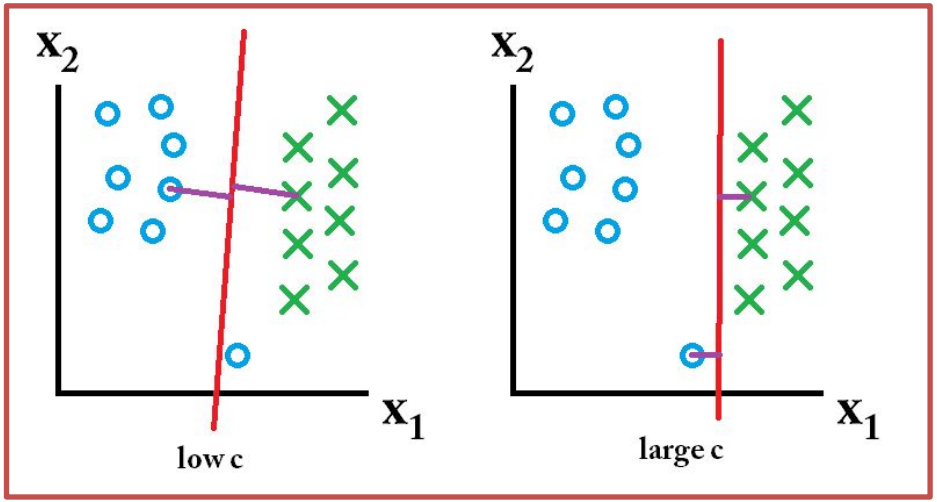
such that: $y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i \ \forall i$

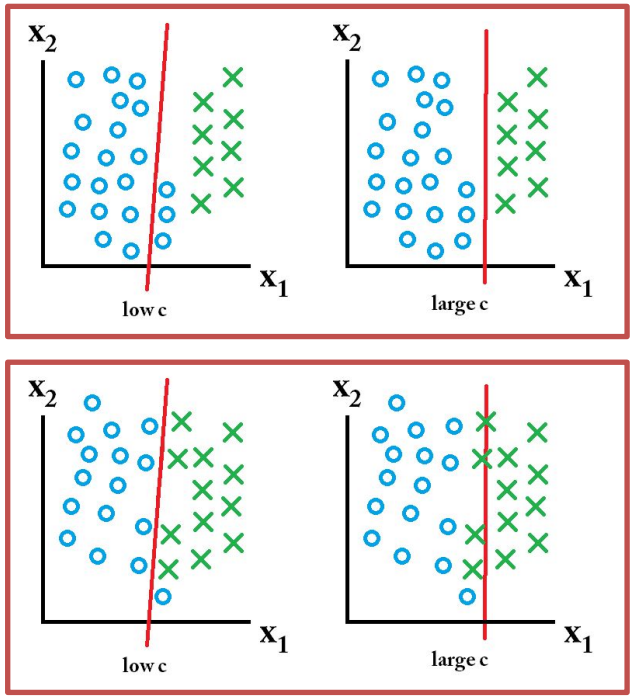
$\xi_i \geq 0 \ \forall i$

Tuning parameter



Finding λ: how soft is too soft?





$c = \lambda^{-1}$ (used by Scikit!)

If $\lambda \to 0$, i.e., $c \to \infty$, we get a hard SVM (like a large Lagrange multiplier)

Part 1:
Kernel methods



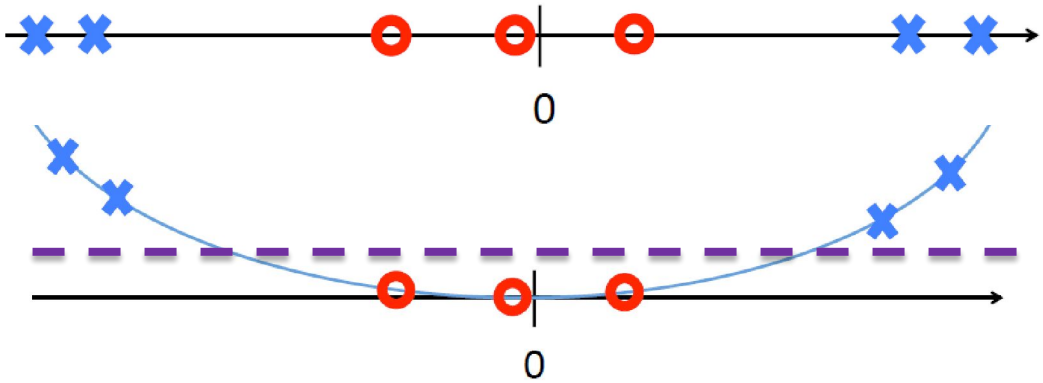
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Moving beyond linear problems

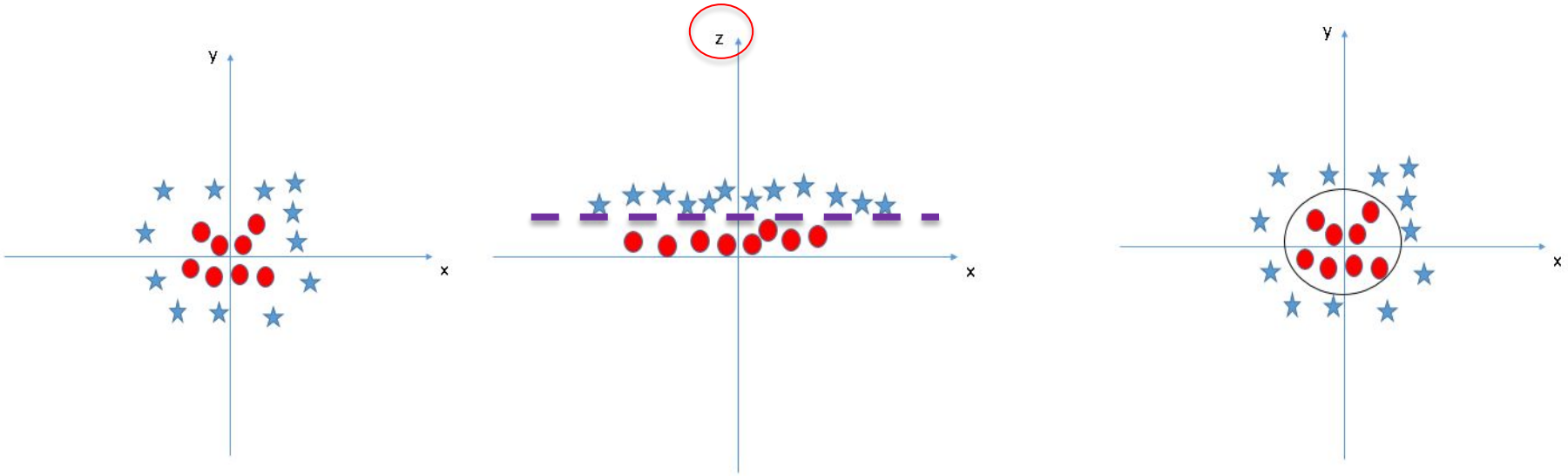
SVMs are powerful, but they are limited to the **linear** case

➔ Can we make non-linear problems linear (like in generalized linear regression?)

- YES! We just need to apply the correct transformation to the input space
- This usually involves increasing the dimension (moving the complexity of the problem)



Example



Non-linear problem → $z = x^2 + y^2$ → Back to the original problem

Embedding into a feature space

We define a non-linear mapping ψ from the input space to a new (usually larger) space $\psi : X \to F$

The new space F is usually $\mathbb{R}^n$, but there is no requirement: it can be a Hilbert space of arbitrary size (even infinite)

We can convert the training set into a sequence $((\psi(\mathbf{x}_1), y_1), \ldots, (\psi(\mathbf{x}_m), y_m))$ and train a **linear predictor** over it

A universal solution?

This sounds like a great idea, but there is a significant issue: complexity has to be preserved somewhere

➔ Increasing the dimension of the space makes halfspaces more expressive
➔ This means increasing the VC dimension (i.e., complexity), which increases the risk of overfitting (the VC dimension is d+1)
➔ There is another problem: computational complexity!

Computing $h(\psi(\mathbf{x}))$ might be very complex if $\psi(\mathbf{x}) \in \mathbb{R}^{10^6}$

The kernel trick

Do we **need** to go through the feature space? Maybe not!

Our kernel function is $K(\mathbf{x}, \mathbf{x}') = \langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle$

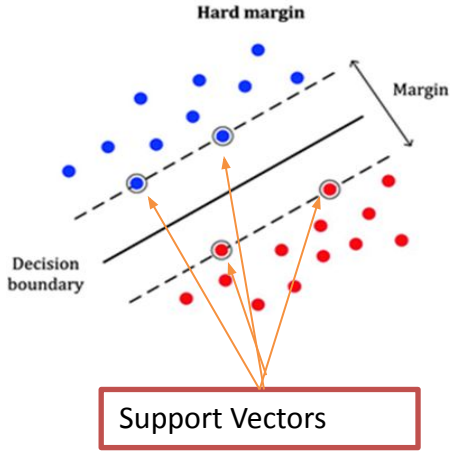
➔ If we can find a simple kernel function (that does not need a conversion to the feature space) we can compute the inner product without increasing computational complexity!

Support vector theorem

The optimal solution $\mathbf{w}_0$ can be represented as a linear combination of the support vectors

Set of support vector indices: $I = \{i \in \{1, \dots, m\} : |\langle \mathbf{w}_0, \mathbf{x}_i \rangle| = 1$

We have $\alpha_i \neq 0$ only if $i \in I$, and $\sum_{i=1}^{m} \alpha_i \mathbf{x}_i = \mathbf{w}_0$



Proof: not part of the course

Back to the hard SVM problem

The hard SVM problem is:

$$\arg\min_{\mathbf{w},b} ||\mathbf{w}||^2$$

$$\text{such that} \quad y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1$$

We can try to write the problem using the theorem:

$$\min_{\mathbf{w}} ||\mathbf{w}||^2 + \max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \sum_{i=1}^{m} \alpha_i (1 - y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b))$$

Infinite if the constraint is violated, 0 otherwise

Finding the dual form (homogeneous rep.)

$$\min_{\mathbf{w}} ||\mathbf{w}||^2 + \max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \sum_{i=1}^{m} \alpha_i (1 - y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle))$$

We can rearrange the problem to become (we add a ½ term that does not matter, as the right term is either 0 or infinite):

$$\min_{\mathbf{w}} \max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \frac{1}{2}||\mathbf{w}||^2 + \sum_{i=1}^{m} \alpha_i (1 - y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle))$$

We can flip the min and max and get an equivalent problem (**strong duality**):

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \min_{\mathbf{w}} \frac{1}{2}||\mathbf{w}||^2 + \sum_{i=1}^{m} \alpha_i (1 - y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle))$$

Simplifying the problem

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \min_{\mathbf{w}} \frac{1}{2} ||\mathbf{w}||^2 + \sum_{i=1}^{m} \alpha_i (1 - y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle)$$

Once α is fixed, the problem is unconstrained, and the objective is convex. The gradient is then:

$$\frac{\partial f}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

Setting the gradient to 0, we get:

$$\min_{\mathbf{w}} f = \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i$$

This is why we added the ½ term, we compensate the 2

We can use the gradient to get a new problem:

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \frac{1}{2} \left\| \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \right\|^2 + \sum_{i=1}^{m} \alpha_i \left(1 - y_i \left\langle \sum_{j=1}^{m} \alpha_j y_j \mathbf{x}_j, \mathbf{x}_i \right\rangle \right)$$

Final step: dual form

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \frac{1}{2} \left\| \sum_{i=1}^{m} \alpha_i y_i \mathbf{x}_i \right\|^2 + \sum_{i=1}^{m} \alpha_i \left(1 - y_i \left\langle \sum_{j=1}^{m} \alpha_j y_j \mathbf{x}_j, \mathbf{x}_i \right\rangle \right)$$

Now we expand the sum and square and get:

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} (\alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle) + \sum_{i=1}^{m} \left(\alpha_i - \alpha_i \sum_{j=1}^{m} \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \right)$$

As the two highlighted terms are the same, we simplify again and obtain:

$$\max_{\alpha \in \mathbb{R}^m : \alpha \geq 0} \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle$$

The input only appears as an inner product!

Representer theorem

The soft SVM problem in feature space can be rewritten as

$$\min_{\mathbf{w}} f(\langle \mathbf{w}, \psi(\mathbf{x}_1) \rangle, \ldots, \langle \mathbf{w}, \psi(\mathbf{x}_m) \rangle) + R(||\mathbf{w}||)$$

Where $R(||\mathbf{w}||)$ is a monotonically non-decreasing function

Assume $\psi : X \to F$ is a mapping from the original feature space to a Hilbert space. There exists a vector $\alpha \in \mathbb{R}^m$ such that $\mathbf{w} = \sum_{i=1}^{m} \alpha_i \psi(\mathbf{x}_i)$ is an optimal solution of the SVM problem in the feature space

Proof (first step)

$$\min_{\mathbf{w}} f(\langle \mathbf{w}, \psi(\mathbf{x}_1) \rangle, \ldots, \langle \mathbf{w}, \psi(\mathbf{x}_m) \rangle) + R(||\mathbf{w}||)$$

Let $\mathbf{w}^*$ be an optimal solution of the SVM problem in the feature space

We can decompose it into a linear combination of components and a remainder

$$\mathbf{w}^* = \sum_{i=1}^{m} \alpha_i \psi(\mathbf{x}_i) + \mathbf{u}$$

The remainder must be orthogonal to all elements

$$\langle \mathbf{u}, \psi(\mathbf{x}_i) \rangle = 0 \forall \mathbf{x}_i \in S$$

Proof (second step)

Set a second weight vector:

$$\mathbf{w}' = \mathbf{w}^* - \mathbf{u}$$

Since $\mathbf{u}$ is orthogonal to all components of $\mathbf{w}'$, we have $\langle \mathbf{u}, \mathbf{w}' \rangle = 0$

We have

$$||\mathbf{w}^*||^2 = ||\mathbf{w}'||^2 + ||\mathbf{u}||^2 \geq ||\mathbf{w}'||^2$$

Since $R(||\mathbf{w}||)$ is monotonically non-decreasing, this means that

$$R(||\mathbf{w}^*||) \geq R(||\mathbf{w}'||)$$

Proof (final step)

Remember that $\langle \mathbf{u}, \mathbf{w}' \rangle = 0$ and $\langle \mathbf{u}, \psi(\mathbf{x}_i) \rangle = 0 \forall \mathbf{x}_i \in S$

We can then say that

$$y_i \langle \mathbf{w}^*, \psi(\mathbf{x}_i) \rangle = y_i \langle \mathbf{w}^* - \mathbf{u}, \psi(\mathbf{x}_i) \rangle = y_i \langle \mathbf{w}', \psi(\mathbf{x}_i) \rangle \ \forall i$$

If the margin is bounded, as $R(||\mathbf{w}^*||) \geq R(||\mathbf{w}'||)$, and the slack variables are the same, $\mathbf{w}' = \mathbf{w}^* - \mathbf{u}$ must also be optimal

We then have at least one optimal solution $\mathbf{w} = \sum_{i=1}^{m} \alpha_i \psi(\mathbf{x}_i)$

Part 2:
Common kernels



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

The polynomial feature space embedding

We use homogeneous coordinates and define $x_0 = 1$

We define $J = \{0, 1, \dots, d\}^N$: it is a vector of degrees of length N, and $\psi_j(\mathbf{x}) = \prod_{i=1}^{d} x_{j_i}$ (basically, we raise each element to the j_i-th power, and multiply everything)

This is equivalent to a Generalized Linear Model: we consider all combinations of powers!

The polynomial kernel

$$K(\mathbf{x}, \mathbf{x}') = (1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N$$

Polynomial kernel

- ➔ The boundary is an N-th degree polynomial
- ➔ The feature space dimension increases by a factor N, and so does VC dimension
- ➔ The parameter N controls the number of bends in the boundary

Proof (first step)

$$K(\mathbf{x}, \mathbf{x}') = (1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N$$

Let us expand the kernel

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \left(\sum_{i=0}^{d} x_i x_i' \right)^N$$

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \prod_{n=1}^{N} \sum_{i=0}^{d} x_i x_i'$$

Proof (second step)

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \prod_{n=1}^{N} \sum_{i=0}^{d} x_i x_i'$$

Let us expand the sum:

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \prod_{n=1}^{N} (1 + x_1 x_1' + x_2 x_2' \ldots + x_d x_d')$$

The product of polynomials is given by (remember that $J = \{0, 1, \ldots, d\}^N$) the combinations of sums

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \sum_{\mathbf{j} \in J} \prod_{i=1}^{d} x_{j_i} x_{j_i}'$$

Proof (final step)

$$(1 + \langle \mathbf{x}, \mathbf{x}' \rangle)^N = \sum_{\mathbf{j} \in J} \prod_{i=1}^{d} x_{j_i} x'_{j_i}$$

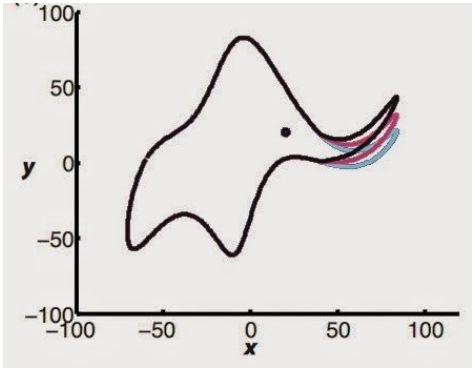
Each element of the feature space embedding is given by $\psi_j(\mathbf{x}) = \prod_{i=1}^{d} x_{j_i}$

We can trivially see that $\langle \psi(\mathbf{x}), \psi(\mathbf{x}') \rangle = K(\mathbf{x}, \mathbf{x}')$

Occam’s razor

All other things being equal, we should choose the simplest explanation (i.e., the smaller and simpler hypothesis class).

“With four parameters I can fit an elephant, and with five I can make him wiggle his trunk." (J. von Neumann)



The RBF kernel

$$K(\mathbf{x}, \mathbf{x}') = e^{-\frac{||\mathbf{x}-\mathbf{x}'||^2}{2\sigma^2}}$$

Radial Basis Function (RBF): Gaussian distribution over the distance!

- ➔ Regions are Gaussian variables
- ➔ We can control the precision (i.e., how close it is to the points) by changing the variance
- ➔ The feature space is infinite-dimensional, and VC dimension is infinite (sample complexity depends on the margin in the feature space)
- ➔ This includes any polynomial predictor in the original space

Proof (1-D input)

We define a function $\psi_n(x) = \frac{1}{\sqrt{n!}} e^{-\frac{x^2}{2}} x^n$

The feature space is infinite-dimensional for $n \in \{0, 1, \ldots, \infty\}$

We have

$$\langle \psi(x), \psi(x') \rangle = \sum_{n=0}^{\infty} \left(\frac{1}{\sqrt{n!}} e^{-\frac{x^2}{2}} x^n \right) \left(\frac{1}{\sqrt{n!}} e^{-\frac{x'^2}{2}} x'^n \right)$$

$$\langle \psi(x), \psi(x') \rangle = e^{-\frac{x^2+x'^2}{2}} \sum_{n=0}^{\infty} \frac{(xx')^n}{n!}$$

Proof (1-D input)

$$\langle \psi(x), \psi(x') \rangle = e^{-\frac{x^2+x'^2}{2}} \sum_{n=0}^{\infty} \frac{(xx')^n}{n!}$$

Remember that the infinite series converges to the exponential function:

$$\sum_{n=0}^{\infty} \frac{x^n}{n!} = e^x$$

$$\langle \psi(x), \psi(x') \rangle = e^{-\frac{x^2+x'^2-2xx'}{2}}$$

$$\langle \psi(x), \psi(x') \rangle = e^{-\frac{||x-x'||^2}{2}}$$

This is equivalent to the RBF kernel function!

Part 3:
Stochastic Gradient Descent

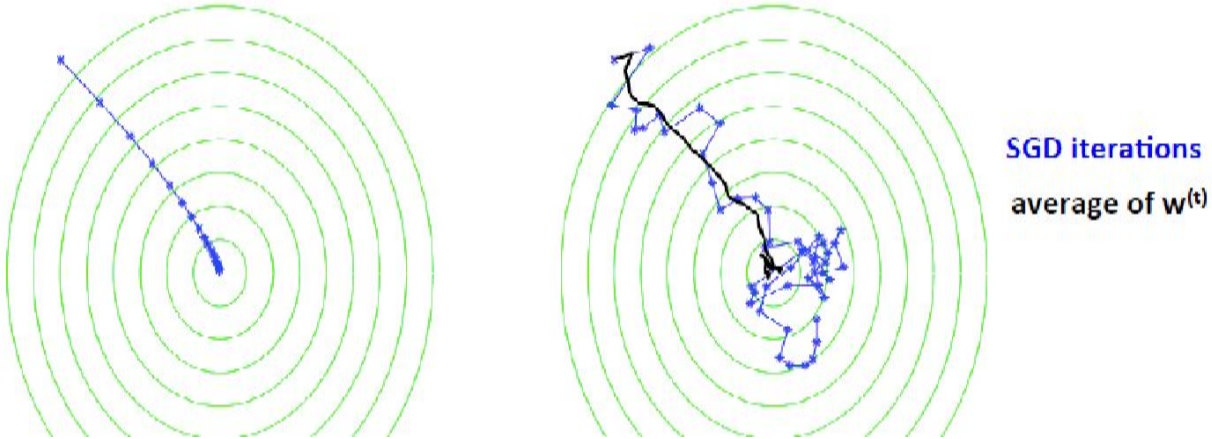


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Stochastic Gradient Descent

GD is computationally expensive: we need to compute the gradient over all points!

We can use a random point (selected from a distribution whose expected value is equal to the gradient) to converge to the same result with a lower cost



Stochastic Gradient Descent

SGD algorithm:

$\boldsymbol{w}^{(0)} \leftarrow \mathbf{0}$

for $t \leftarrow 0$ to $T-1$ do

choose $\boldsymbol{v}_t$ at random from a distribution such that $\mathbb{E}\left[\boldsymbol{v}_t \middle| \boldsymbol{w}^{(t)}\right] = \nabla f(\boldsymbol{w}^{(t)})$

$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \boldsymbol{v}_t$

return $\bar{\boldsymbol{w}} = \boldsymbol{w}^{(T)}$ (or $\bar{\boldsymbol{w}} = \frac{1}{T-T_0} \sum_{t=T_0}^{T} \boldsymbol{w}^{(t)}$)

The cost of each iteration is reduced by a factor m

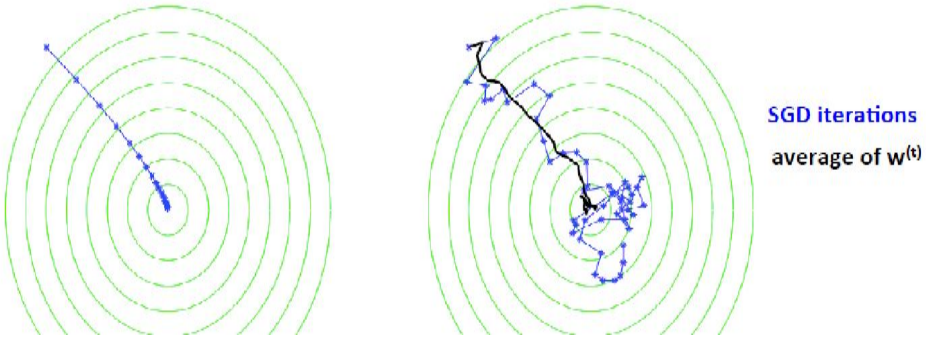
We can apply the same principle in the case of regularized loss and other risk minimization frameworks (e.g., SRM)

Stochastic Gradient Descent

SGD can get unstuck if there are local minima in non-convex cases

Usually, the final point is computed as the average over multiple iterations after approximate convergence

We can use mini-batches of k samples to strike a balance between GD and SGD: the parameter can be used to tune the stability of the algorithm



SGD for risk minimization

Stochastic Gradient Descent (SGD) for minimizing $L_D(\boldsymbol{w})$

params: Scalar $\eta > 0$, integer $T > 0$

Init: $\boldsymbol{w}^{(1)} = \mathbf{0}$
for $t = 1,2,\ldots,T$
    sample $z \sim D$
    pick $\boldsymbol{v_t} = \nabla \ell(\boldsymbol{w}^{(t)}, z)$
    update $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - \eta \boldsymbol{v_t}$
output $\boldsymbol{w}^{(T)}$

SGD finds an **unbiased** estimate of the gradient by picking a fresh sample (it can also be used for RLM)

SGD bound for risk minimization

If the problem is ρ-Lipschitz and the optimal weights have $||\mathbf{w}^*|| \leq B$, then for any value of ε we get that, after T iterations, SGD can approximate any learning problem:

$$T \geq \frac{B^2\rho^2}{\varepsilon^2} \wedge \eta = \frac{B}{\rho\sqrt{T}} \implies \mathbb{E}[L_D(\bar{\mathbf{w}})] \leq \min_{\mathbf{w}\in H} L_D(\mathbf{w}) + \varepsilon$$

This is a result of the bound on the performance of GD

SGD bound for RLM

Regularized Loss Minimization Problem (Tikhonov):

$$\min_{\mathbf{w}} \left(\frac{\lambda}{2} ||\mathbf{w}||^2 + L_S(||\mathbf{w}||) \right)$$

The regularized loss is λ-strongly convex: we can use a step size $\eta_t = \frac{1}{\lambda t}$ (details in the book, not part of the course)

If the loss is ρ-Lipschitz, after T iterations we get

$$\mathbb{E}[f(\bar{\mathbf{w}})] - f(\mathbf{w}^*) \leq \frac{4\rho^2}{\lambda T}(1 + \log(T))$$

SGD bound for RLM: proof (first step)

The update rule is:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\lambda \mathbf{w}^{(t)} + \mathbf{v}_t}{\lambda t}$$

We can group the two weights to get

$$\mathbf{w}^{(t+1)} = \frac{t-1}{t}\mathbf{w}^{(t)} - \frac{\mathbf{v}_t}{\lambda t}$$

Then, we apply the update rule again:

$$\mathbf{w}^{(t+1)} = \frac{t-1}{t}\left(\frac{t-2}{t-1}\mathbf{w}^{(t-1)} - \frac{\mathbf{v}_{t-1}}{\lambda(t-1)}\right) - \frac{\mathbf{v}_t}{\lambda t}$$

SGD bound for RLM: proof (second step)

We start from the two-step update rule we computed:

$$\mathbf{w}^{(t+1)} = \frac{t-1}{t}\left(\frac{t-2}{t-1}\mathbf{w}^{(t-1)} - \frac{\mathbf{v}_{t-1}}{\lambda(t-1)}\right) - \frac{\mathbf{v}_t}{\lambda t}$$

We can simplify the fractions to get

$$\mathbf{w}^{(t+1)} = \frac{t-2}{t}\mathbf{w}^{(t-1)} - \frac{\mathbf{v}_t + \mathbf{v}_{t-1}}{\lambda t}$$

If we apply the rule recursively, we get

$$\mathbf{w}^{(t+1)} = -\frac{1}{\lambda t}\sum_{k=1}^{t}\mathbf{v}_k$$

This is equivalent to the gradient over the whole dataset

SGD for soft SVM

We want to solve

$$\min_{\mathbf{w}} \left(\frac{\lambda}{2} ||\mathbf{w}||^2 + \frac{1}{m} \sum_{i=1}^{m} \max\{0, 1 - y\langle \mathbf{w}, \mathbf{x}_i \rangle\} \right)$$

Variant of SGD for RLM

$$\boldsymbol{w}^{(t+1)} = -\frac{1}{\lambda t} \sum_{j=1}^{t} \boldsymbol{v}_j$$

Note: it's standard to add a $\frac{1}{2}$ in the regularization term to simplify some computations.

Algorithm:

$\theta^{(1)} \leftarrow \mathbf{0}$;

for $t \leftarrow 1$ *to* T **do**

  let $\mathbf{w}^{(t)} \leftarrow \frac{1}{\lambda t} \theta^{(t)}$;

  choose i uniformly at random from $\{1, \ldots, m\}$;

  if $y_i \langle \mathbf{w}^{(t)}, \mathbf{x}_i \rangle < 1$ **then** $\theta^{(t+1)} \leftarrow \theta^{(t)} + y_i \mathbf{x}_i$;

  else $\theta^{(t+1)} \leftarrow \theta^{(t)}$;

return $\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{w}^{(t)}$;

$$\theta^{(t)} = -\sum_{j=1}^{t-1} \boldsymbol{v}_j$$

Hinge loss: gradient is 0 if correctly classified and $-y_i x_i$ if error

SGD for soft SVMs (with kernels)

SGD for Solving Soft-SVM with Kernels

Goal: Solve Equation (16.5)
parameter: T
Initialize: $\boldsymbol{\beta}^{(1)} = \mathbf{0}$
for $t = 1, \ldots, T$
  Let $\boldsymbol{\alpha}^{(t)} = \frac{1}{\lambda t}\boldsymbol{\beta}^{(t)}$
  Choose i uniformly at random from $[m]$
  For all $j \neq i$ set $\beta_j^{(t+1)} = \beta_j^{(t)}$
  If $(y_i \sum_{j=1}^m \alpha_j^{(t)} K(\mathbf{x}_j, \mathbf{x}_i) < 1)$
    Set $\beta_i^{(t+1)} = \beta_i^{(t)} + y_i$
  Else
    Set $\beta_i^{(t+1)} = \beta_i^{(t)}$
Output: $\bar{\mathbf{w}} = \sum_{j=1}^m \bar{\alpha}_j \psi(\mathbf{x}_j)$ where $\bar{\boldsymbol{\alpha}} = \frac{1}{T}\sum_{t=1}^T \boldsymbol{\alpha}^{(t)}$

$$\boldsymbol{\theta}^{(t)} = \sum_{j=1}^m \beta_j^{(t)} \psi(x_j)$$

$$\boldsymbol{w}^{(t)} = \sum_{j=1}^m \alpha_j^{(t)} \psi(x_j)$$

The representer theorem coefficients yield the same results as a standard soft SVM over the feature space

SGD issues

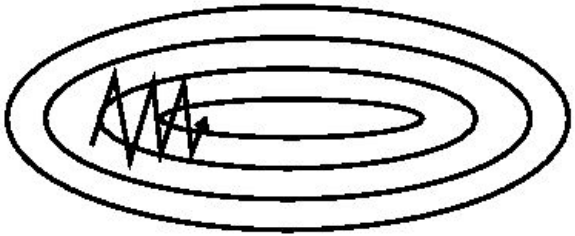
- ➔ The learning rate is a critical problem: trade-off between convergence speed (high rate) and stability (low rate)
- ➔ Adaptive learning rates (such as the RLM rule) need to be defined in advance and might not always work
- ➔ Some of the inputs may require different rates (the rate is too high in one dimension and too low in another)

SGD with momentum

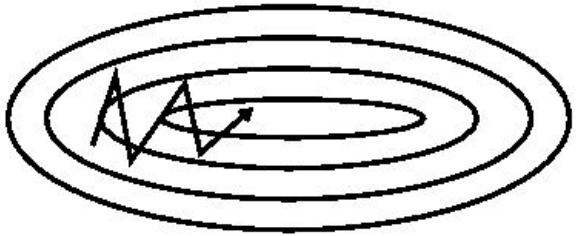
SGD tends to oscillate if one dimension is much steeper than the other (common around local optima). This can be solved by introducing **momentum**, i.e., combining previous gradients (usually with $\gamma \simeq 0.9$)

$$\mathbf{v}_t = \gamma \mathbf{v}_{t-1} + (1-\gamma)\nabla L(\mathbf{w}_t)$$

Momentum is like rolling a sphere downhill: as it rolls, it accumulates momentum, and thus tends to keep going in the same direction, with fewer oscillations



SGD without momentum



SGD with momentum

Adaptive schemes for SGD

- ➔ **Adagrad** adapts the learning rate for each parameter independently, with higher rates for parameters associated with features that appear rarely
- ➔ **Adam** (adaptive moment estimation) combines Adagrad and momentum, behaving like a rolling ball with friction: it prefers flatter minima, with some regularization

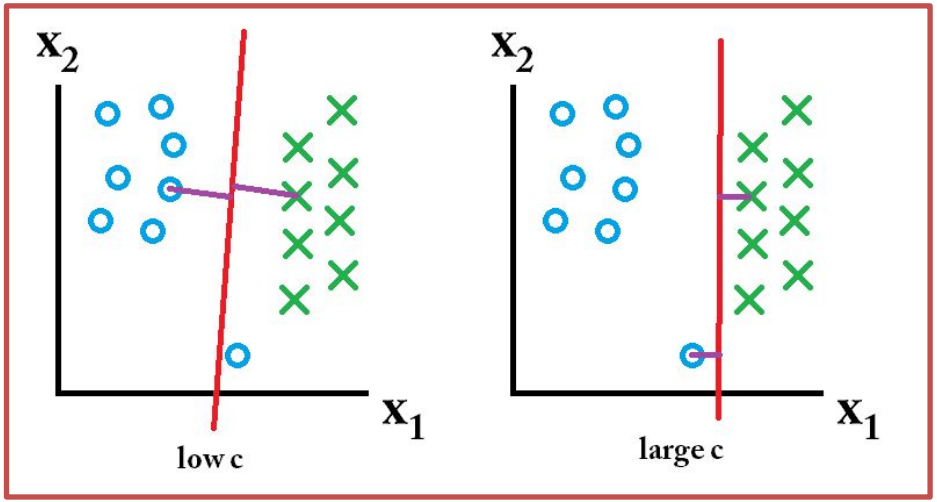
Adam is the most common method in neural network training, but there are multiple variants. The optimizer can be seen as another component of the learning algorithm, and thus selected with cross-validation!

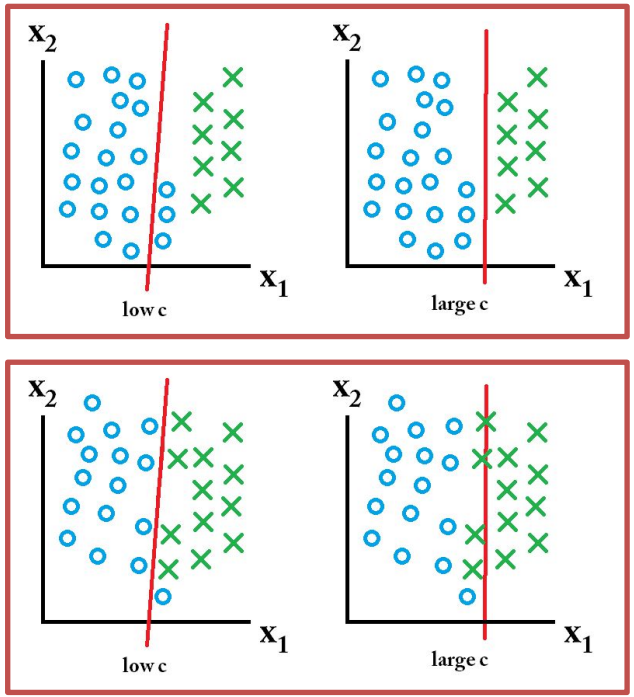
Part 4:
Practical SVMs and exercises



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Finding λ: how soft is too soft?

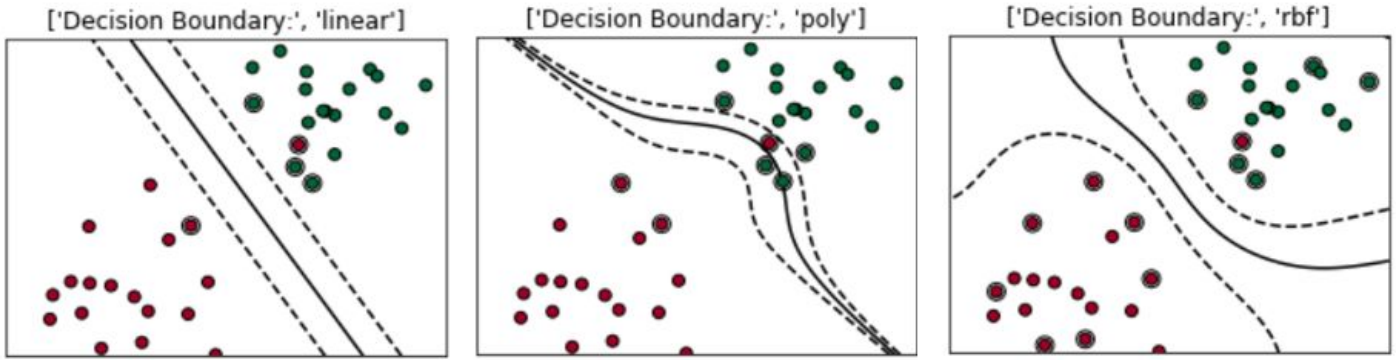




$c = \lambda^{-1}$ (used by Scikit!)

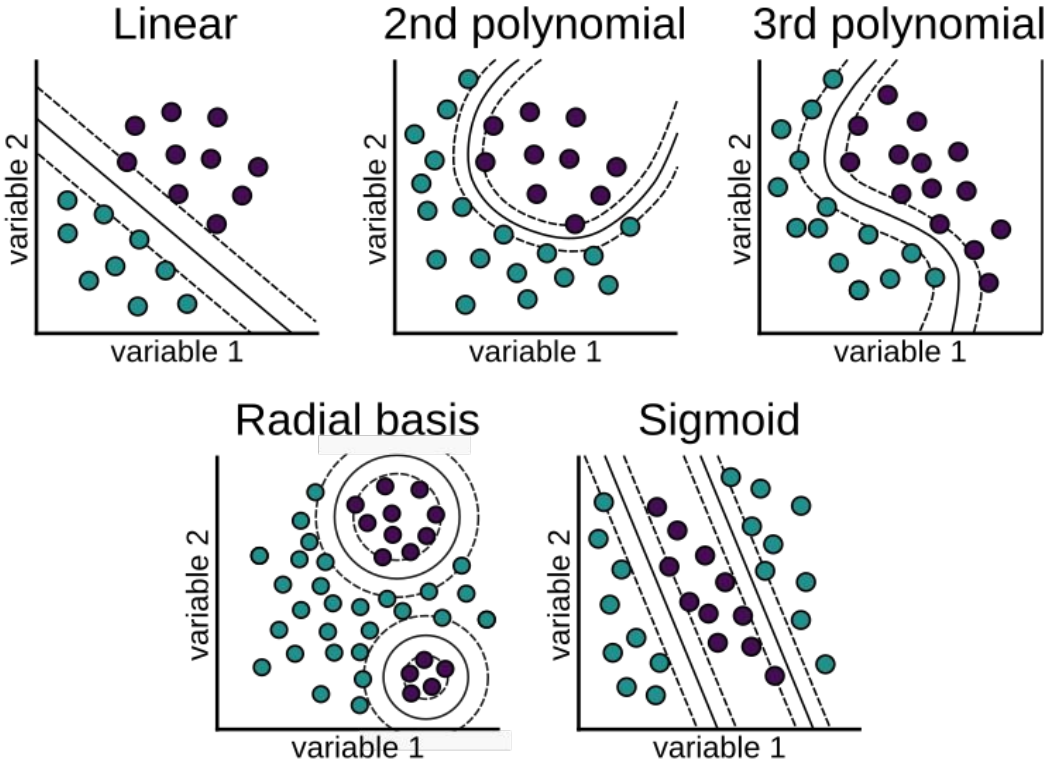
If $\lambda \to 0$, i.e., $c \to \infty$, we get a hard SVM (like a large Lagrange multiplier)

Using different kernels



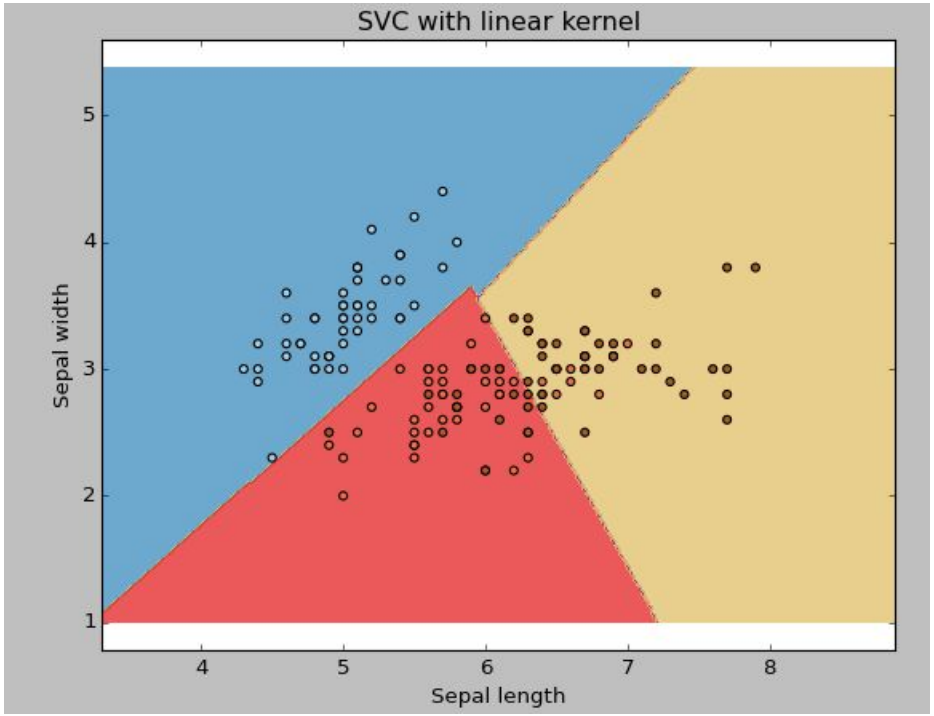
images from towardsdatascience and R-Bloggers

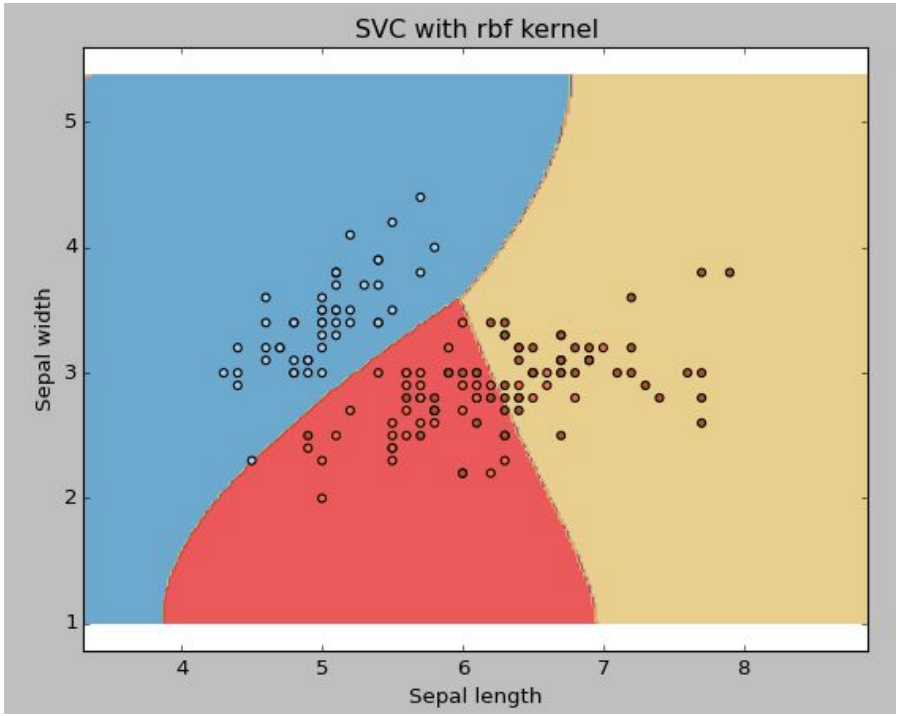
Using different kernels



images from towardsdatascience and R-Bloggers

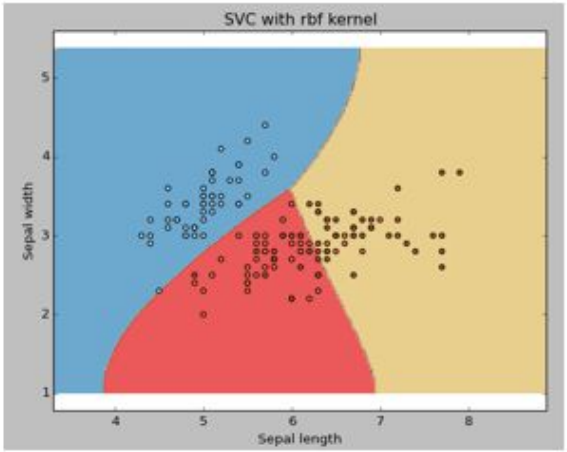
The RBF kernel



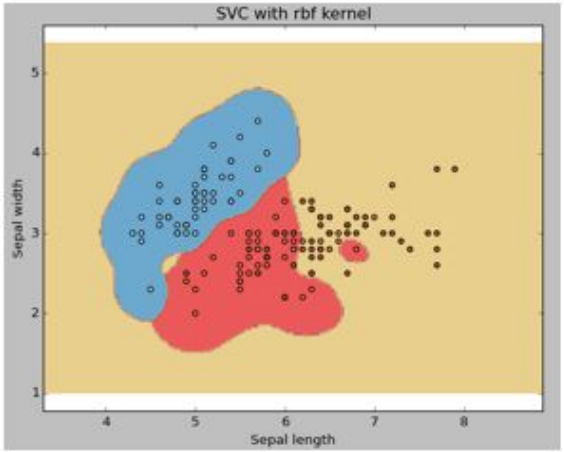


The RBF kernel

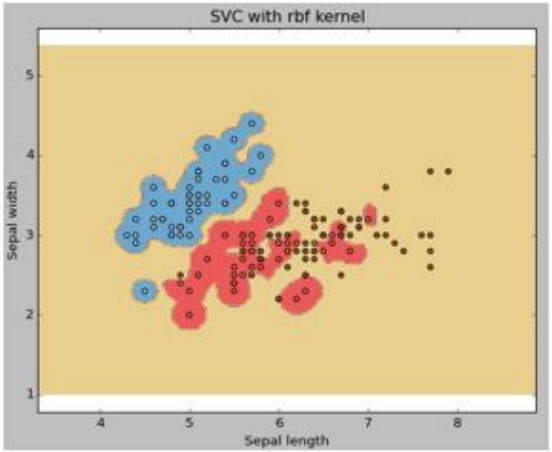
gamma =0



gamma =10

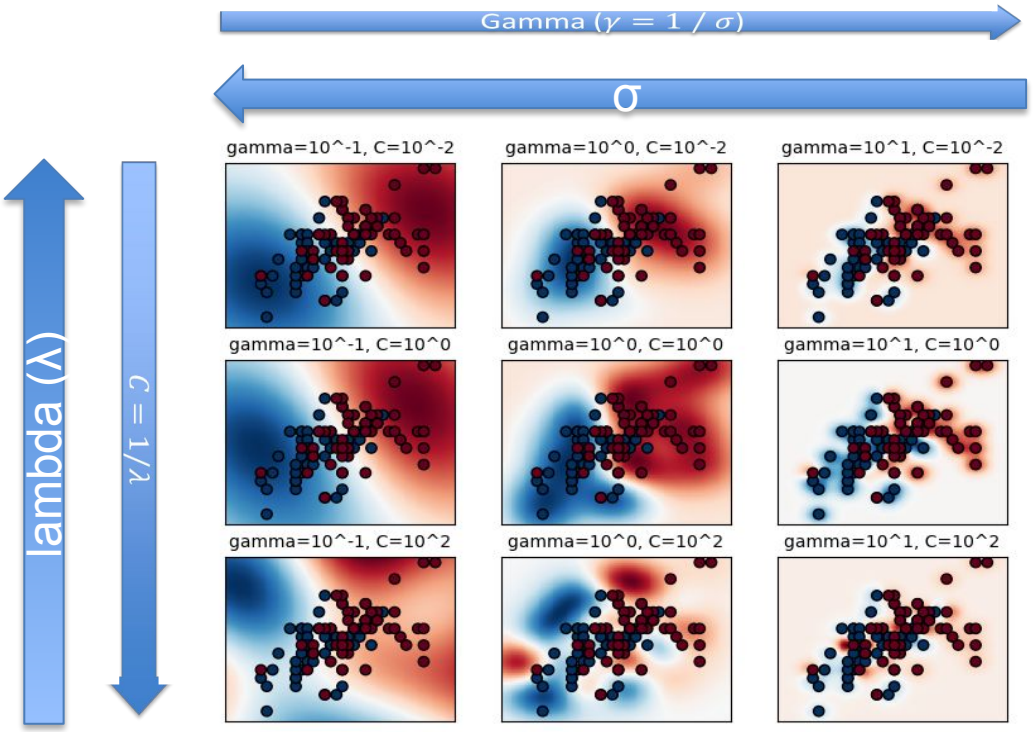


gamma =100



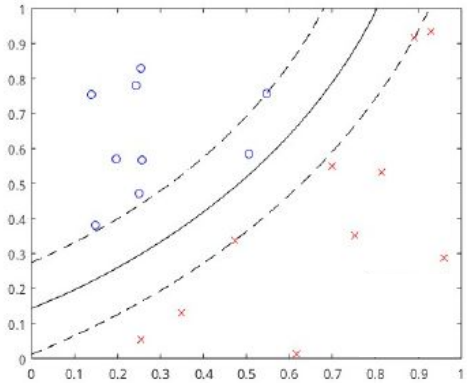
The standard deviation σ (γ=1/σ is often used in libraries) controls the precision of the fit: a lower standard deviation means that the border

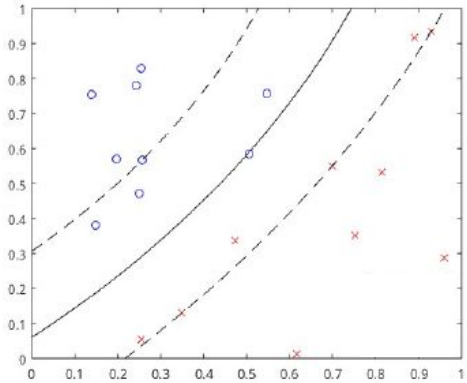
Practical SVMs: σ and λ with the RBF kernel

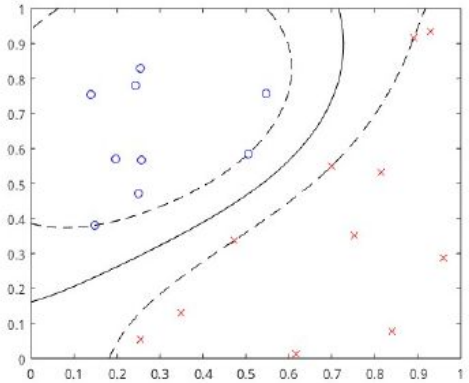


images from sklearn documentation

Exercise: recognizing parameters







a) $\sigma=2, \lambda=0.1$
b) $\sigma=2, \lambda=0.01$
c) $\sigma=0.5, \lambda=0.01$

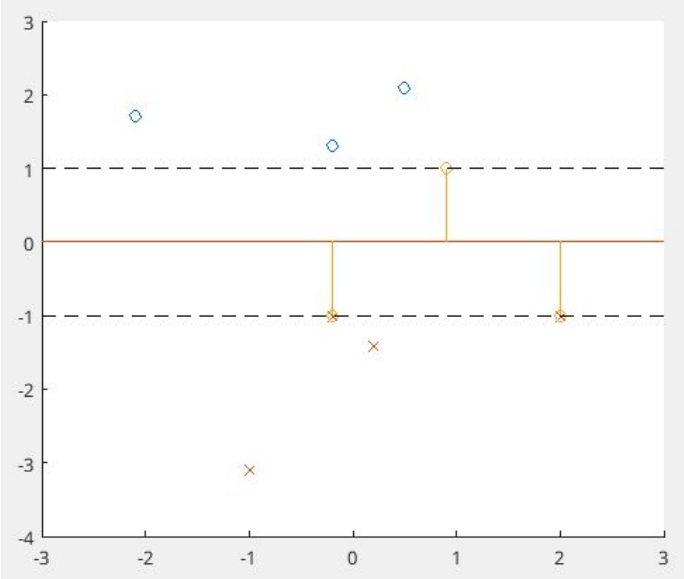
Exercise: support vectors

Index	x	y	α
1	(0.2,-1.4)	-1	0
2	(-2.1,1.7)	1	0
3	(0.9,1)	1	0.5
4	(-1,-3.1)	-1	0
5	(-0.2,-1)	-1	0.25
6	(-0.2,1.3)	1	0
7	(2,-1)	-1	0.25
8	(0.5,2.1)	1	0

a) What are the support vectors?
b) Can you draw the plot?
c) Would it be possible, by moving only 2 data points, to get the same hyperplane with only 2 support vectors? Draw the approximate solution

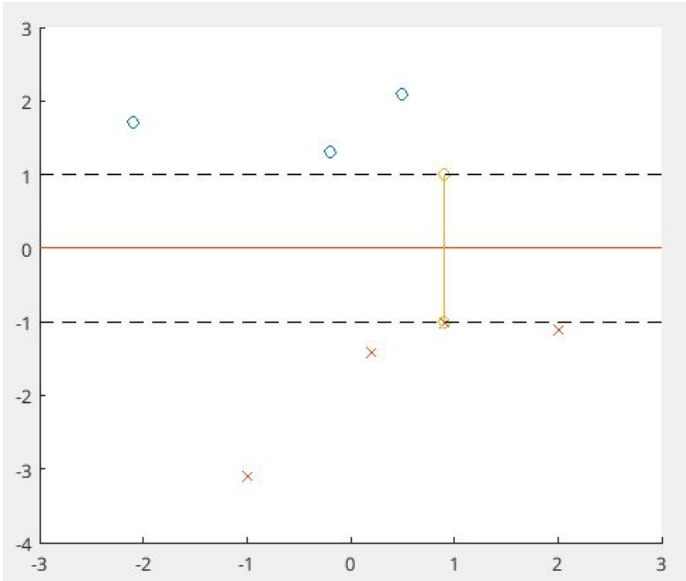
Exercise: solution

Index	x	y	α
1	(0.2,-1.4)	-1	0
2	(-2.1,1.7)	1	0
3	(0.9,1)	1	0.5
4	(-1,-3.1)	-1	0
5	(-0.2,-1)	-1	0.25
6	(-0.2,1.3)	1	0
7	(2,-1)	-1	0.25
8	(0.5,2.1)	1	0



Exercise: solution

Index	x	y	α
1	(0.2,-1.4)	-1	0
2	(-2.1,1.7)	1	0
3	(0.9,1)	1	0.5
4	(-1,-3.1)	-1	0
5	(-0.2,-1)	-1	0.25
6	(-0.2,1.3)	1	0
7	(2,-1)	-1	0.25
8	(0.5,2.1)	1	0

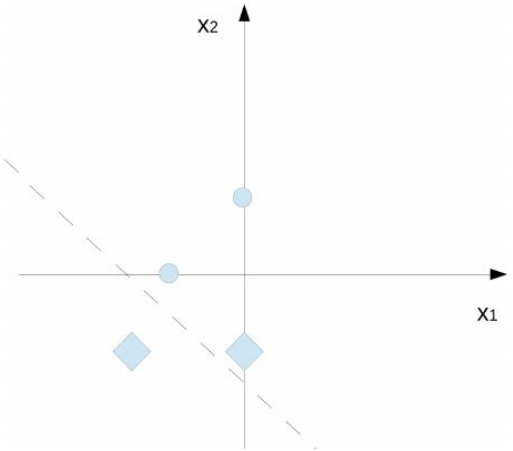


Part 5: Exercises



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

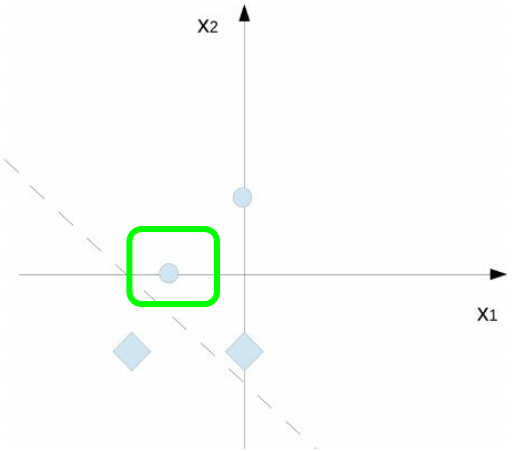
Exercise 2.2



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

The initial weight vector is (2,1.5,1.5) and the learning rate is 1

Checking the points

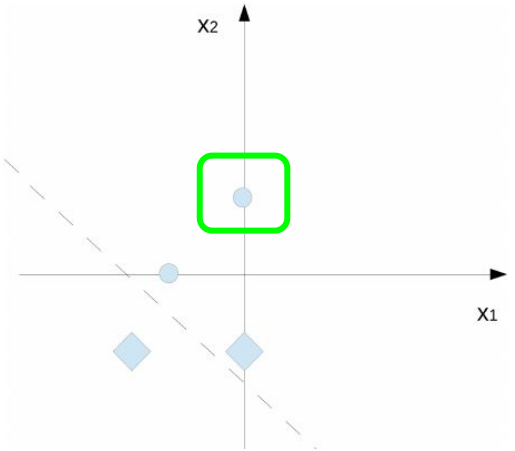


x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

2x1: bias
1.5x(-1): first coordinate
1.5x0: second coordinate
total:2-1.5=0.5

Checking the points

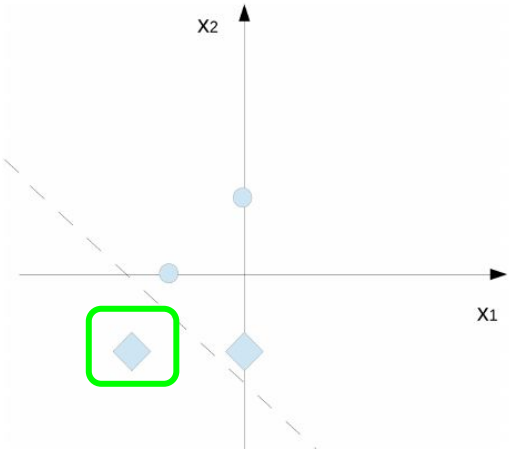


x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

2x1: bias
1.5x0: first coordinate
1.5x1: second coordinate
total:2+1.5=3.5

Checking the points

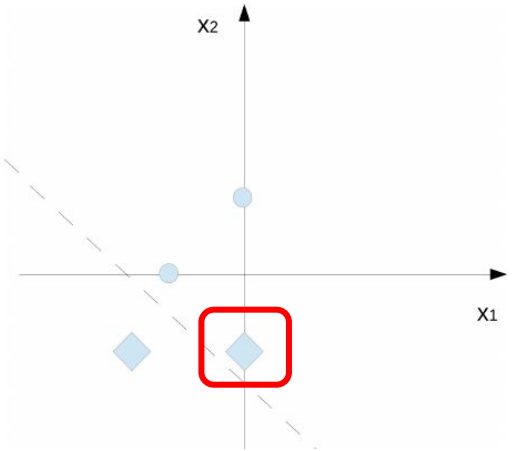


x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

2x1: bias
1.5x(-1.5): first coordinate
1.5x(-1): second coordinate
total:2-2.25-1.5=-1.75

Checking the points

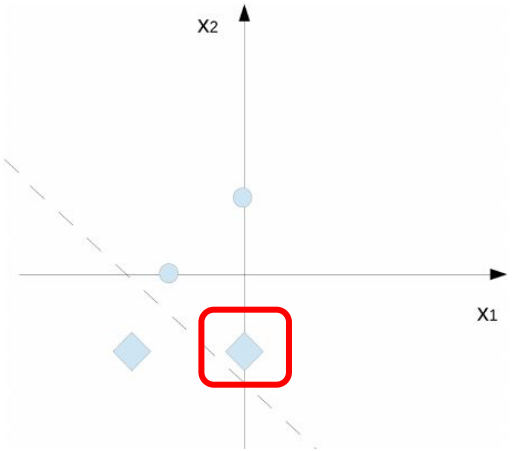


x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

2x1: bias
1.5x0: first coordinate
1.5x(-1): second coordinate
total:2-1.5=0.5

Perceptron update



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w^{(1)}=w^{(0)}+y_i x_i$

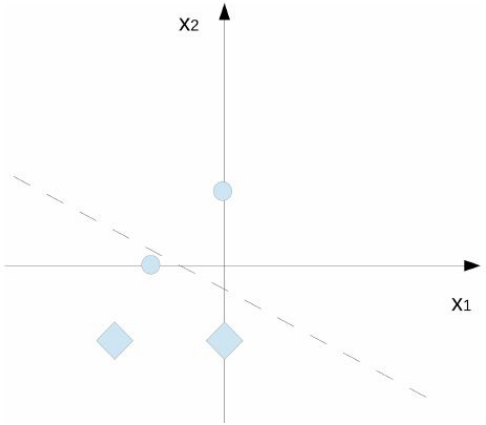
2-1: bias

1.5-0: first coord.

1.5-(-1): second coord.

$w^{(1)}=(1,1.5,2.5)$

Drawing the new boundary



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

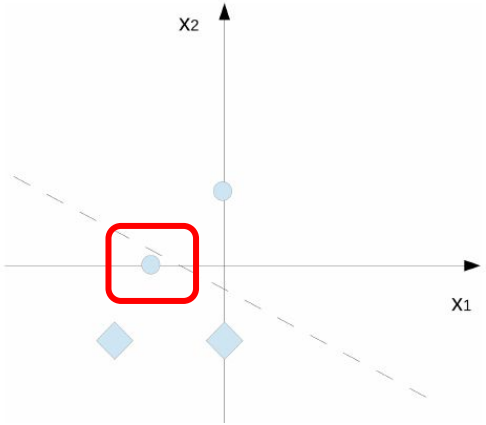
$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w_0+w_1x_1+w_2x_2=0$

$x_2=-(w_1/w_2)x_1-(w_0/w_2)$ -> $x_2=-3/5x_1-2/5$

Checking the points



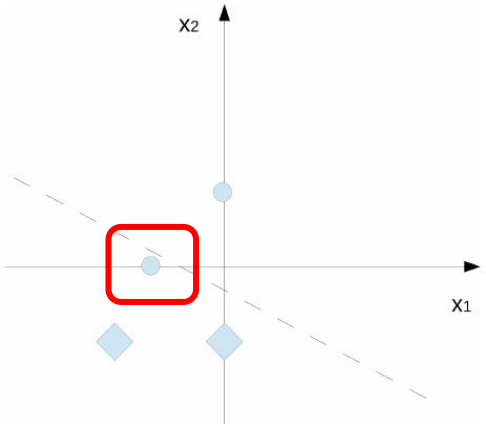
x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

1x1: bias
1.5x(-1): first coordinate
2.5x0: second coordinate
total:1-1.5=-0.5

Perceptron update



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w^{(2)}=(2,0.5,2.5)$

$w^{(2)}=w^{(1)}+y_i x_i$

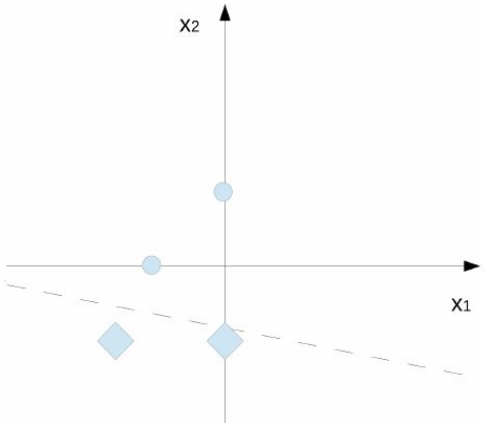
1+1: bias

1.5+(-1): first coord.

2.5+0: second coord.

$w^{(2)}=(2,0.5,2.5)$

Drawing the new boundary



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

$w^{(0)}=(2,1.5,1.5)$

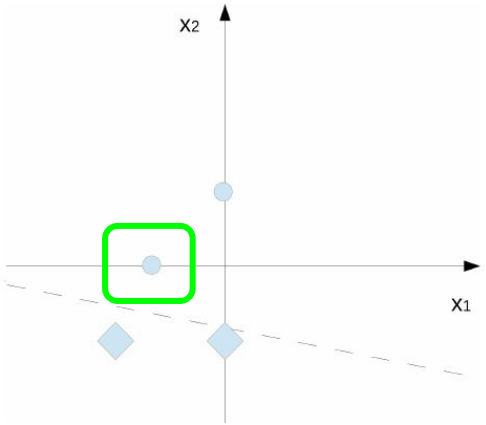
$w^{(1)}=(1,1.5,2.5)$

$w^{(2)}=(2,0.5,2.5)$

$w_0+w_1x_1+w_2x_2=0$

$x_2=-(w_1/w_2)x_1-(w_0/w_2)$ -> $x_2=-1/5x_1-4/5$

Checking the points



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

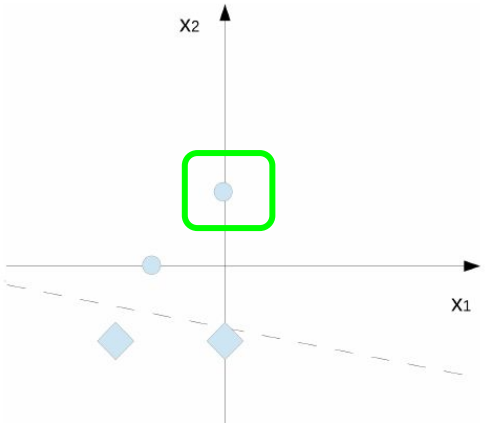
$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w^{(2)}=(2,0.5,2.5)$

2x1: bias
0.5x(-1): first coordinate
2.5x0: second coordinate
total:2-0.5+0=1.5

Checking the points



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

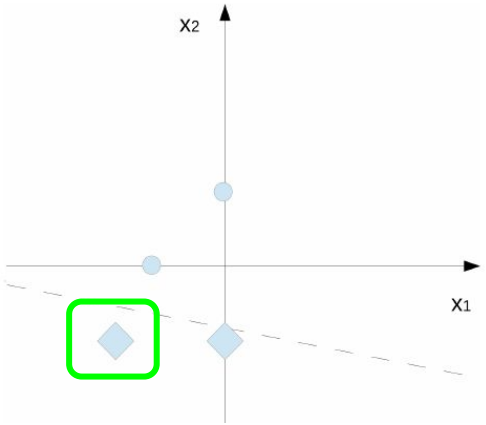
$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w^{(2)}=(2,0.5,2.5)$

2x1: bias
0.5x0: first coordinate
2.5x1: second coordinate
total:2+0+2.5=4.5

Checking the points



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

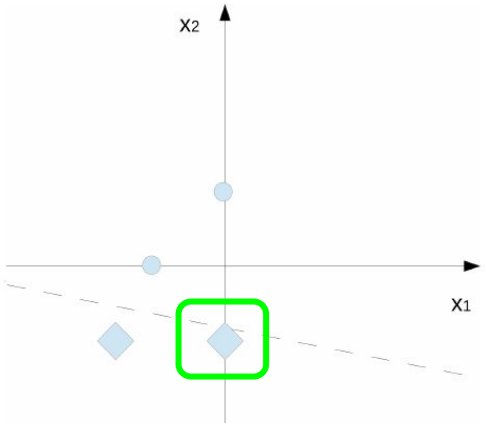
$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

$w^{(2)}=(2,0.5,2.5)$

2x1: bias
0.5x(-1.5): first coordinate
2.5x(-1): second coordinate
total:2-0.75-2.5=-1.25

Checking the points



x_1	x_2	y
-1	0	1
0	1	1
-1.5	-1	-1
0	-1	-1

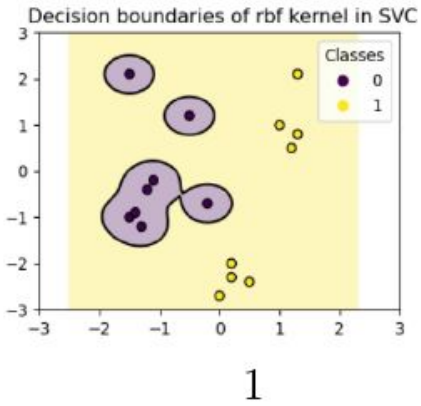
$w^{(0)}=(2,1.5,1.5)$

$w^{(1)}=(1,1.5,2.5)$

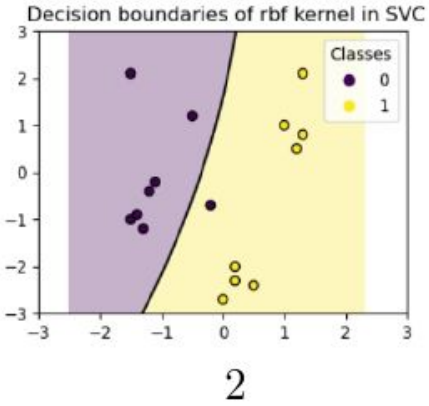
$w^{(2)}=(2,0.5,2.5)$

2x1: bias
0.5x0: first coordinate
2.5x(-1): second coordinate
total:2+0-2.5=-0.5

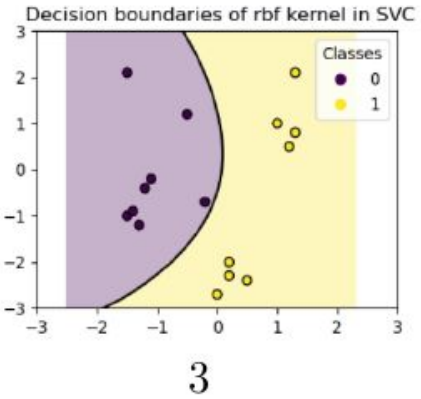
Exercise 2.3



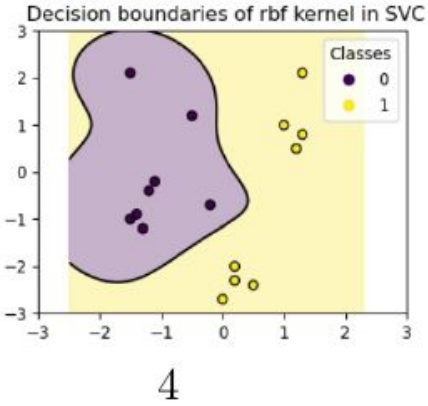
1



2



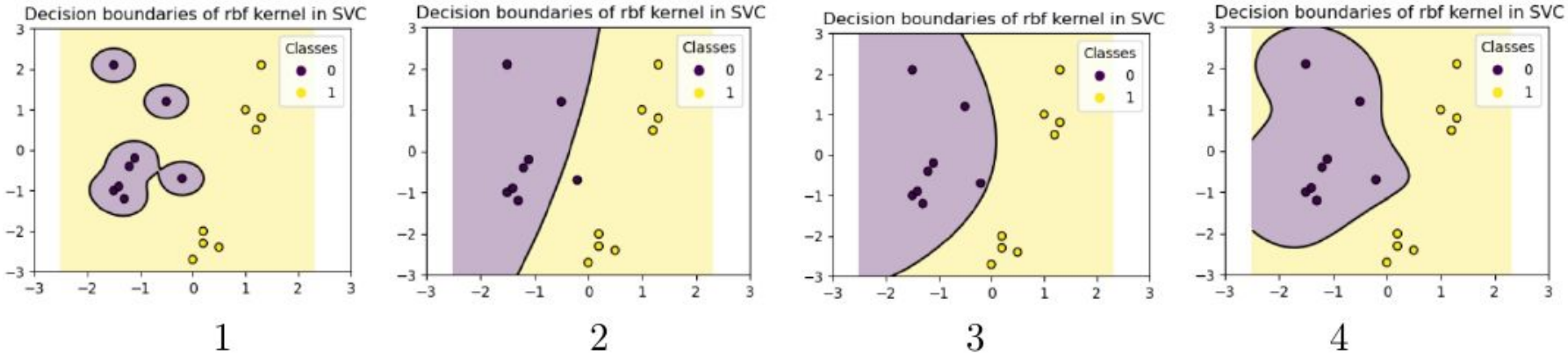
3



4

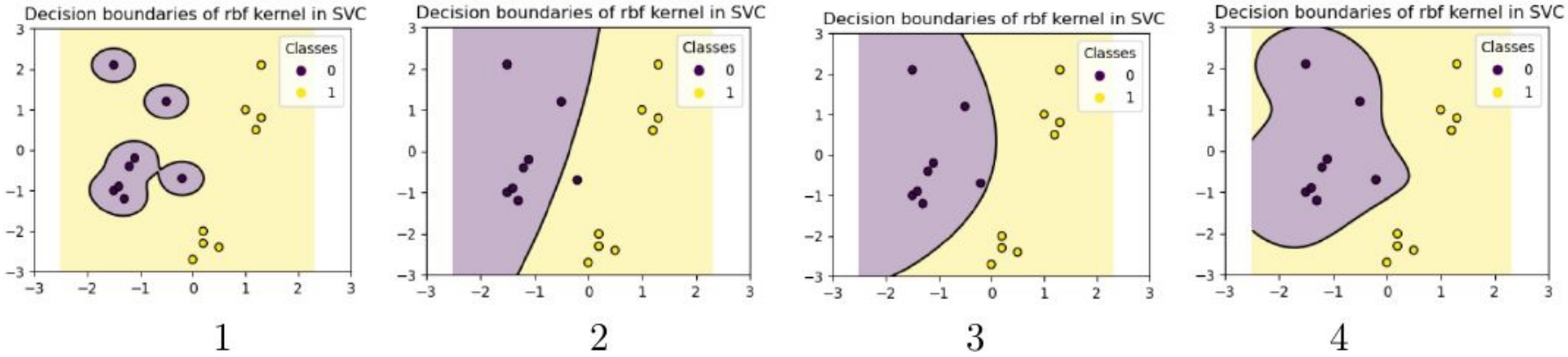
a) γ=0.01
b) γ=0.1
c) γ=1
d) γ=10

Finding the smallest variance



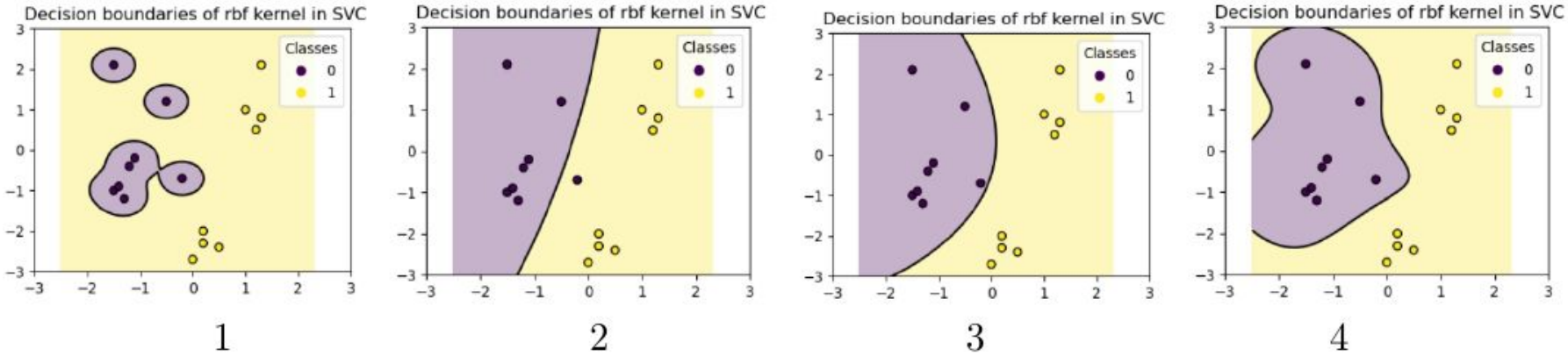
Let us look at image 1: it is the one with the tightest curves in the boundary. This means that σ is small, so γ must be big: we can assign it to γ=10, i.e., setting d

Finding the biggest variance



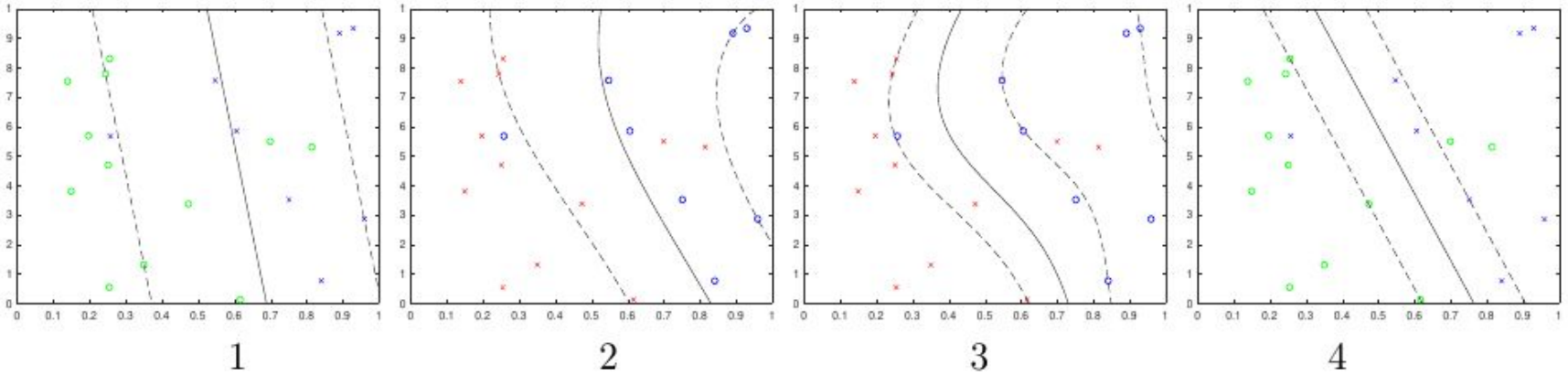
The second image is the one with the widest curve, so it must have the largest value of σ, i.e., the smallest γ (case a, γ=0.01)

Intermediate cases



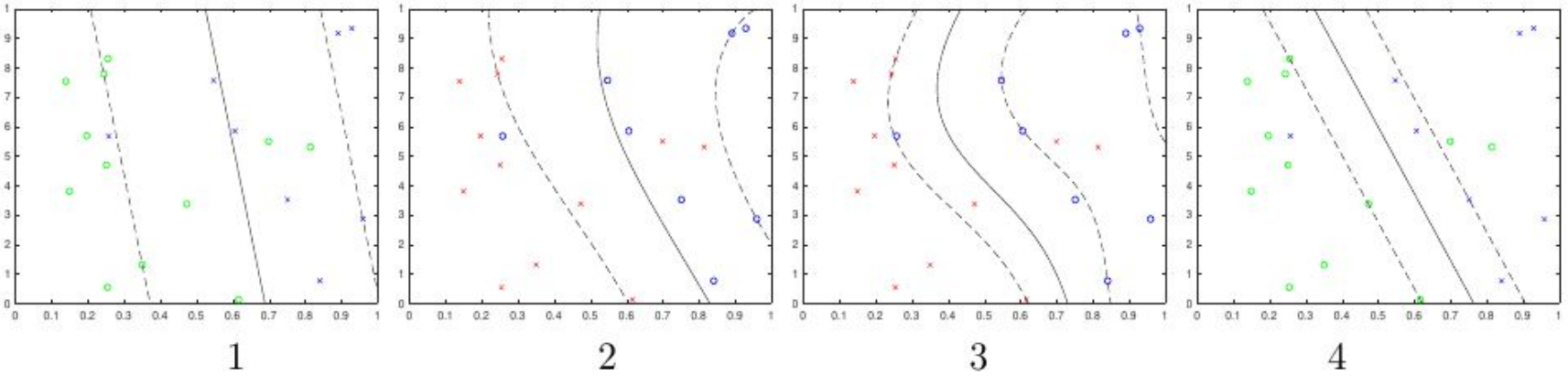
The third image has a wider curve than the fourth, so it must have a larger value of σ: by exclusion, we assign the third image to case b (γ=0.1) and the fourth image to case c (γ=0.1). The correct mapping is DABC

Exercise 2.4



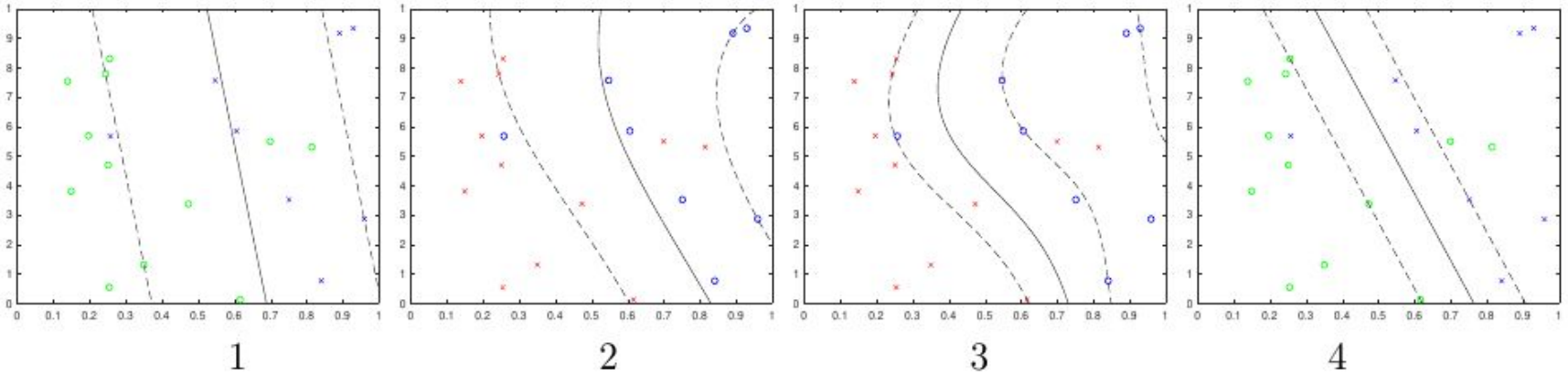
a) Linear kernel, C=100
b) Linear kernel, C=10
c) RBF kernel, C=10, σ=1
d) RBF kernel, C=100, σ=1

Identifying the kernel



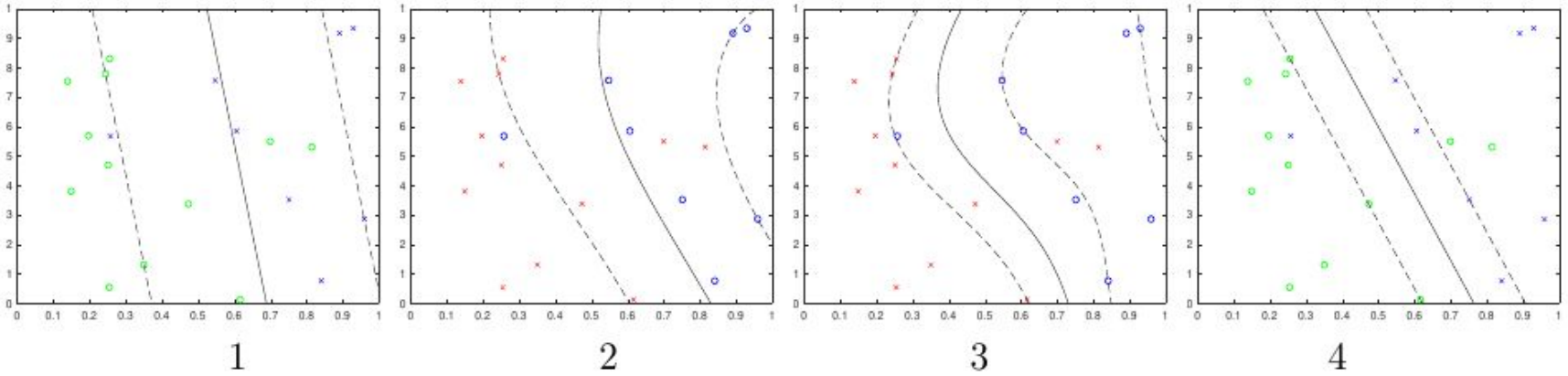
First, we can identify the kernel: images 1 and 4 have linear boundaries. 2 and 3 must then correspond to the RBF kernel

Linear kernel: considering the margin



The first image has a wide margin, with 8 outliers. The fourth image has 3 outliers and a much narrower margin. We then assign image 1 to the case with a larger value of λ, i.e., a smaller C. Image 1 then corresponds to case b, while image 4 corresponds to case a.

RBF kernel: considering the margin



The second image has a relatively wide margin, with 9 outliers. The third image has 4 outliers and a much narrower margin. We then assign image 2 to the case with a larger value of λ, i.e., a smaller C. Image 2 then corresponds to case c, while image 3 corresponds to case d. The final pattern is BCDA